

Implementación del algoritmo de búsqueda de Grover

Ejecución en simulador y en hardware cuántico real de IBM

Lenguajes de Programación

Ingeniería de Sistemas y Computación

Universidad Nacional de Colombia

Autor: Maicol Sebastián Olarte Ramírez

SDK utilizado: Qiskit 2.5.2 · Backend: ibm_marrakesh (Heron r2, 156 qubits)

Fecha: 20 de septiembre de 2026

Tabla de contenido

1. Marco teórico.....	4
1.1 Qubit, superposición y medición.....	4
1.2 Puertas cuánticas empleadas.....	4
1.3 El problema de búsqueda no estructurada.....	4
1.4 Amplificación de amplitud: cómo funciona.....	5
1.5 Número óptimo de iteraciones y sobre-rotación.....	5
1.6 Hardware cuántico actual: la era NISQ.....	5
2. Descripción y justificación del problema.....	7
2.1 Descripción del problema.....	7
2.2 Justificación.....	7
2.3 Alcance y limitaciones.....	8
3. Diseño de la solución.....	9
3.1 Diagrama de bloques.....	9
3.2 Pseudocódigo.....	9
3.3 Diseño del oráculo.....	10
3.4 Diseño del difusor.....	11
3.5 Circuito completo.....	11
4. Código fuente y dependencias.....	12
4.1 Dependencias y versiones utilizadas.....	12
4.2 Estructura de archivos entregados.....	12
4.3 Código fuente completo: grover.py.....	12
5. Manual de usuario y manual técnico.....	15
5.1 Manual de usuario.....	15
5.2 Manual técnico.....	16
6. Experimentación y análisis de resultados.....	18
6.1 Escenario E1 — Simulador ideal.....	18
6.2 Escenario E2 — Simulador con ruido realista.....	18
6.3 Escenario E3 — Computador cuántico real de IBM.....	19
6.4 Escenario E4 — Barrido del número de iteraciones.....	20
6.5 Escenario E5 — Generalidad del oráculo.....	21
6.6 Síntesis de resultados.....	22
7. Conclusiones.....	23
8. Referencias.....	23

Correspondencia con los literales del enunciado

La siguiente tabla indica en qué sección de este documento se desarrolla cada uno de los ítems solicitados:

Literal	Ítem solicitado	Sección	Pág.
a	Marco teórico	1	4
b	Descripción y justificación del problema a resolver	2	7
c	Diseño de la aplicación (diagrama de bloques y pseudocódigo)	3	9
d	Código fuente completo y versiones de las librerías	4	12
e	Manual de usuario y manual técnico	5	15
f	Experimentación y análisis para al menos 3 escenarios de prueba (se documentan 5)	6	18

1. Marco teórico

Corresponde al literal (a) del enunciado.

1.1 Qubit, superposición y medición

La unidad de información de la computación clásica es el bit, que vale 0 o 1. La unidad de la computación cuántica es el qubit, cuyo estado general es una combinación lineal — una superposición — de los dos estados base:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \quad \text{con} \quad |\alpha|^2 + |\beta|^2 = 1$$

Los coeficientes α y β se llaman amplitudes y son números complejos. Mientras el qubit no se mide, ambos términos coexisten. Al medirlo el estado colapsa a $|0\rangle$ o a $|1\rangle$ con probabilidades $|\alpha|^2$ y $|\beta|^2$ respectivamente, y la superposición se pierde.

Con n qubits el sistema vive en un espacio de 2^n estados base y el estado general es una superposición de todos ellos. Con 3 qubits, por ejemplo, se representan simultáneamente los 8 estados $|000\rangle \dots |111\rangle$. Esta es la fuente del paralelismo cuántico: una sola operación actúa sobre las 2^n amplitudes a la vez.

La dificultad central: ese paralelismo no es gratis. La medición devuelve un único resultado, elegido al azar según las probabilidades. Evaluar una función sobre los 2^n valores a la vez no sirve de nada si al medir sale uno cualquiera. Un algoritmo cuántico útil debe, antes de medir, *concentrar* la probabilidad en la respuesta correcta. Eso es exactamente lo que hace Grover.

1.2 Puertas cuánticas empleadas

Las operaciones sobre qubits son matrices unitarias, llamadas puertas. Las que usa este trabajo son:

Puerta	Acción	Uso en el algoritmo
H (Hadamard)	Crea superposición: $ 0\rangle \rightarrow (0\rangle + 1\rangle)/\sqrt{2}$	Estado inicial y construcción del difusor
X (NOT)	Intercambia $ 0\rangle \leftrightarrow 1\rangle$	Adapta el oráculo a cualquier objetivo
Z	Cambia el signo de $ 1\rangle$	Base del marcado por fase
MCX / Toffoli	X sobre el último qubit si todos los controles valen 1	Núcleo del oráculo y del difusor
CZ	Fase -1 si ambos qubits valen 1	Puerta nativa de dos qubits en los chips Heron

Truco de la Z multicontrolada: no existe una puerta "MCZ" nativa, pero se obtiene rodeando una MCX de dos Hadamard sobre el qubit objetivo: $H \cdot MCX \cdot H = MCZ$. Esta identidad aparece dos veces en el código, en el oráculo y en el difusor.

1.3 El problema de búsqueda no estructurada

Se tiene una colección de N elementos sin ningún orden ni estructura aprovechable, y una función f que solo sabe decir, para un elemento dado, si es el buscado ($f(x)=1$) o no ($f(x)=0$). A esa función se le llama oráculo, y el costo del algoritmo se mide en número de consultas al oráculo.

Sin estructura no hay nada mejor que probar uno por uno: en el peor caso N consultas, en promedio $N/2$. Es decir, complejidad $O(N)$. Este límite es demostrable: ningún algoritmo clásico puede hacerlo mejor.

El resultado de Grover (1996): un algoritmo cuántico resuelve el mismo problema con $O(\sqrt{N})$ consultas. Para $N = 1.000.000$ son ~ 1.000 consultas en vez de ~ 500.000 . La ventaja es cuadrática — no exponencial como en el algoritmo de Shor — pero se aplica a un problema muchísimo más general, y además está demostrado que $O(\sqrt{N})$ es óptimo: ningún algoritmo cuántico puede superarlo.

1.4 Amplificación de amplitud: cómo funciona

El algoritmo parte de la superposición uniforme, donde los N estados tienen la misma amplitud $1/\sqrt{N}$, y repite un bloque de dos operaciones que juntas constituyen una iteración de Grover:

- **1) Oráculo (marcado por fase).** Invierte el signo de la amplitud del estado buscado y deja los demás intactos: $|x\rangle \rightarrow -|x\rangle$ si x es la solución. Nótese que esto no cambia ninguna probabilidad ($|-a|^2 = |a|^2$); el estado marcado sigue siendo indistinguible si se midiera en ese momento. El oráculo solo prepara el terreno.
- **2) Difusor (inversión sobre el promedio).** Refleja cada amplitud respecto al promedio de todas: $a \rightarrow 2\bar{a} - a$. Como la amplitud del marcado quedó negativa, el promedio $\bar{a}$ baja ligeramente; al reflejar, la amplitud negativa salta a un valor positivo grande, mientras que las demás bajan un poco.

El resultado neto de cada iteración es que la amplitud del elemento buscado crece a costa de las otras. Geométricamente, el estado vive en un plano de dos dimensiones (el eje de la solución y el eje de todo lo demás) y cada iteración lo rota un ángulo fijo θ hacia el eje de la solución, con $\sin(\theta/2) = 1/\sqrt{N}$.

1.5 Número óptimo de iteraciones y sobre-rotación

Tras k iteraciones la probabilidad de medir la solución es exactamente:

$$P(k) = \sin^2 \left((2k+1) \cdot \arcsin(1/\sqrt{N}) \right)$$

El número de iteraciones que maximiza esa expresión es:

$$k_{\text{óptimo}} = \lfloor (\pi/4) \cdot \sqrt{N} \rfloor$$

Consecuencia importante: Grover no converge monótonamente. Pasado el óptimo, la rotación continúa y el estado se aleja del eje de la solución, con lo que la probabilidad de acierto *baja*. Es un algoritmo donde iterar de más es tan perjudicial como iterar de menos. El escenario E4 de la sección 6 mide este fenómeno experimentalmente.

1.6 Hardware cuántico actual: la era NISQ

Los procesadores disponibles hoy se describen como NISQ (Noisy Intermediate-Scale Quantum): tienen entre decenas y unos cientos de qubits, pero no implementan corrección de errores. Tres limitaciones condicionan cualquier ejecución real:

- **Conjunto de puertas nativo.** El chip no ejecuta puertas arbitrarias. Los procesadores Heron de IBM solo implementan rz , sx , x y cz . Cualquier otra puerta —incluidas H y Toffoli— debe descomponerse en esas cuatro.
- **Conectividad limitada.** Los qubits físicos forman una red donde cada uno solo interactúa con sus vecinos inmediatos. Si el algoritmo necesita una puerta entre dos qubits no vecinos, hay que insertar operaciones SWAP para acercarlos, lo que añade más puertas todavía.

- **Error y decoherencia.** Cada puerta de dos qubits introduce un error del orden de 10^{-3} , y los qubits pierden su estado en decenas o cientos de microsegundos (tiempos T_1 y T_2). Cuanto más profundo el circuito, más se degrada el resultado.

Transpilación. El proceso de traducir el circuito lógico al conjunto nativo y a la topología concreta del chip se llama transpilación, y es automático en Qiskit. Como se verá en la sección 6, es la principal causa de la pérdida de precisión en hardware real: multiplica por seis la profundidad del circuito de este trabajo.

2. Descripción y justificación del problema

Corresponde al literal (b) del enunciado.

2.1 Descripción del problema

Se plantea implementar y ejecutar el algoritmo de búsqueda de Grover sobre un espacio de 8 elementos, codificados con 3 qubits como las cadenas $|000\rangle$ a $|111\rangle$, de forma que el programa identifique un elemento marcado previamente elegido — en esta implementación $|101\rangle$ — usando el mínimo número de consultas al oráculo.

Formalmente, dado el oráculo

$$f(x) = 1 \text{ si } x = 101 ; \quad f(x) = 0 \text{ en cualquier otro caso}$$

se busca determinar el valor de x que satisface $f(x)=1$, sin ninguna información adicional sobre f más allá de poder evaluarla.

El programa debe, además, ejecutarse en tres entornos distintos — simulador ideal, simulador con ruido y computador cuántico real de IBM — y comparar los resultados, de modo que la diferencia entre la teoría y el hardware disponible quede documentada con datos propios.

2.2 Justificación

Por qué este problema y no otro. La búsqueda no estructurada es el caso de uso más representativo de la computación cuántica por tres razones:

- **Es universal.** Muchísimos problemas se reducen a "buscar una aguja en un pajar": invertir una función hash, resolver SAT por fuerza bruta, encontrar una clave criptográfica, buscar en una base de datos sin índice, resolver problemas de optimización combinatoria. Grover acelera todos ellos de forma genérica, sin necesidad de que el problema tenga estructura matemática especial.
- **La ventaja está demostrada y es óptima.** A diferencia de otros algoritmos cuya superioridad depende de conjeturas, aquí la cota $O(\sqrt{N})$ está probada tanto por arriba (Grover la alcanza) como por abajo (nadie puede mejorarla). Es un resultado sólido, no una promesa.
- **Es didácticamente ideal.** Con solo 3 qubits y 2 iteraciones el algoritmo cabe en un circuito que se puede dibujar entero, ejecutar en hardware real gratuito en menos de un minuto, y cuyo resultado correcto se reconoce a simple vista en el histograma. Ilustra los dos conceptos centrales —superposición y amplificación de amplitud— sin la complejidad matemática de Shor.

Relevancia práctica inmediata. Grover tiene un impacto directo en criptografía: reduce a la mitad la seguridad efectiva de los cifrados simétricos (romper AES-128 pasaría de 2^{128} a 2^{64} operaciones), razón por la cual los estándares actuales recomiendan migrar a AES-256. Entender el algoritmo es entender por qué esa recomendación existe.

Por qué ejecutarlo en hardware real y no solo simularlo. Un simulador siempre confirma la teoría, y por tanto no enseña nada sobre el estado actual de la tecnología. Ejecutar en un chip físico obliga a enfrentarse a la transpilación, al ruido y a la decoherencia, y permite cuantificar con números propios la distancia entre el algoritmo sobre el papel y el algoritmo en una máquina de 2026. Ese contraste es el aporte principal de este trabajo.

2.3 Alcance y limitaciones

Aspecto	Decisión adoptada	Motivo
Tamaño	3 qubits (8 elementos)	Circuito legible y ejecutable en el plan gratuito de IBM
Soluciones	Una sola ($ 101\rangle$)	Simplifica el cálculo del número óptimo de iteraciones
Oráculo	Construido conociendo el objetivo	Estándar en la enseñanza de Grover: el circuito no "consulta" el valor, lo codifica
Mitigación de error	No se aplica	Se busca medir el ruido tal cual es, no ocultarlo
Repeticiones	1024 shots por ejecución	Error estadístico $\approx 1.5\%$, suficiente para las conclusiones

Sobre el oráculo. Cabe aclarar una objeción habitual: el circuito del oráculo se construye sabiendo cuál es el objetivo, por lo que en este ejemplo concreto la "búsqueda" es circular. Esto es una convención didáctica universal. En una aplicación real el oráculo sería un circuito que evalúa una condición — por ejemplo, "este candidato es una solución válida del sudoku" — sin que nadie conozca de antemano qué entrada la satisface. Lo que se demuestra aquí es el mecanismo de amplificación, que es idéntico en ambos casos.

3. Diseño de la solución

Corresponde al literal (c) del enunciado.

El diseño se presenta en tres niveles de detalle: un diagrama de bloques del flujo general, el pseudocódigo del algoritmo, y el diseño detallado de los dos subcircuitos que constituyen el núcleo (oráculo y difusor).

3.1 Diagrama de bloques

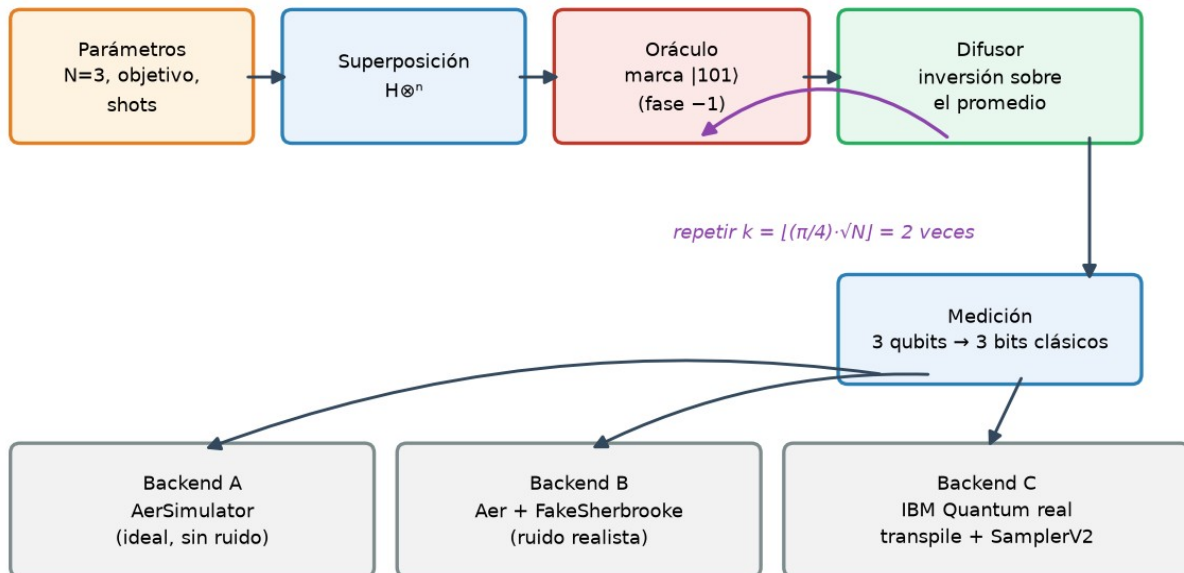


Figura 1. Flujo general de la solución. El bloque oráculo + difusor se repite $k = 2$ veces antes de medir; el mismo circuito se envía a tres backends distintos.

El diseño separa deliberadamente la construcción del circuito de su ejecución. El circuito se arma una sola vez y es agnóstico al backend; los tres modos de ejecución (ideal, con ruido y hardware real) reciben exactamente el mismo objeto `QuantumCircuit`. Esto garantiza que las diferencias observadas en la sección 6 se deban únicamente al entorno de ejecución y no a variaciones en el algoritmo.

3.2 Pseudocódigo

```
ALGORITMO Grover(n, objetivo, shots)
  ENTRADA : n          - número de qubits
            objetivo    - cadena binaria de n bits que se desea encontrar
            shots       - número de repeticiones del experimento
  SALIDA  : distribución de frecuencias de los  $2^n$  estados

  N <-  $2^n$ 
  k <- piso( (pi/4) * raiz(N) )      // iteraciones óptimas

  crear circuito con n qubits y n bits clásicos

  // 1. Superposición uniforme: los N elementos, todos a la vez
  PARA cada qubit q HACER aplicar H(q)

  // 2. Amplificación de amplitud
```

```

REPETIR k VECES
  ORACULO(objetivo)
  DIFUSOR()
FIN REPETIR

// 3. Colapso
medir los n qubits en los n bits clásicos
RETORNAR ejecutar(circuito, shots)
FIN

SUBROUTINA ORACULO(objetivo)          // marca |objetivo> con fase -1
  ceros <- posiciones donde objetivo tiene un '0'
  SI ceros no está vacío ENTONCES aplicar X en esas posiciones
  aplicar Z-multicontrolada sobre los n qubits // = H · MCX · H
  SI ceros no está vacío ENTONCES aplicar X en esas posiciones // deshacer
FIN

SUBROUTINA DIFUSOR()                  // inversión sobre el promedio
  aplicar H en todos los qubits
  aplicar X en todos los qubits
  aplicar Z-multicontrolada sobre los n qubits
  aplicar X en todos los qubits
  aplicar H en todos los qubits
FIN

```

3.3 Diseño del oráculo

El oráculo debe invertir la fase únicamente del estado objetivo. La Z multicontrolada disponible marca de forma natural solo el estado $|11\dots 1\rangle$, así que se emplea la siguiente estrategia de tres pasos:

- Aplicar X en las posiciones donde el objetivo tiene un 0. Esto transforma temporalmente el estado objetivo en $|111\rangle$.
- Aplicar la Z multicontrolada, que ahora marca justamente el estado deseado.
- Aplicar de nuevo las mismas X para deshacer el cambio y devolver los demás estados a su posición original.

Para el objetivo $|101\rangle$ el bit central es el único cero, de modo que se aplica X sobre el qubit 1 antes y después de la MCZ:

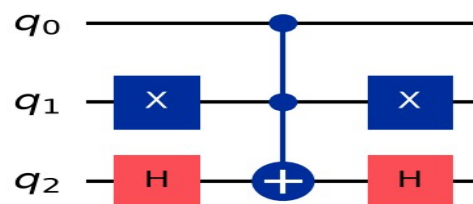


Figura 2. Circuito del oráculo para el objetivo $|101\rangle$. La secuencia H-MCX-H sobre el qubit 2 implementa la Z multicontrolada.

Detalle de implementación — orden de los bits. Qiskit numera los qubits en orden *little-endian*: el qubit 0 corresponde al bit de menor peso, es decir al carácter de la *derecha* de la cadena. Por eso el código recorre la cadena objetivo invertida (*reversed*) al calcular las posiciones de los ceros. Ignorar este detalle produce un programa que busca el estado espejado y es uno de los errores más frecuentes al implementar Grover.

3.4 Diseño del difusor

El difusor implementa la reflexión sobre el estado de superposición uniforme, que equivale a la operación $2|s\rangle\langle s| - I$. Se construye en tres capas simétricas: las Hadamard trasladan el problema a la base computacional, las X más la Z multicontrolada realizan una reflexión respecto a $|00\dots 0\rangle$, y las capas finales deshacen el cambio de base.

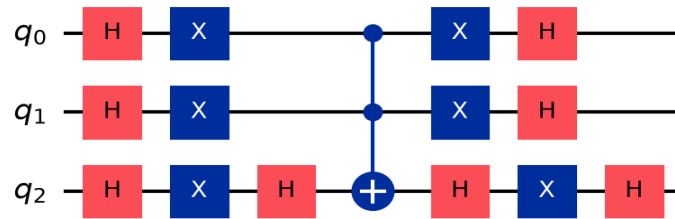


Figura 3. Circuito del difusor. Es independiente del objetivo: el mismo bloque sirve para cualquier búsqueda.

3.5 Circuito completo

Ensamblando los bloques anteriores, el circuito final consta de la capa inicial de Hadamard, dos repeticiones del par oráculo-difusor, y la medición:

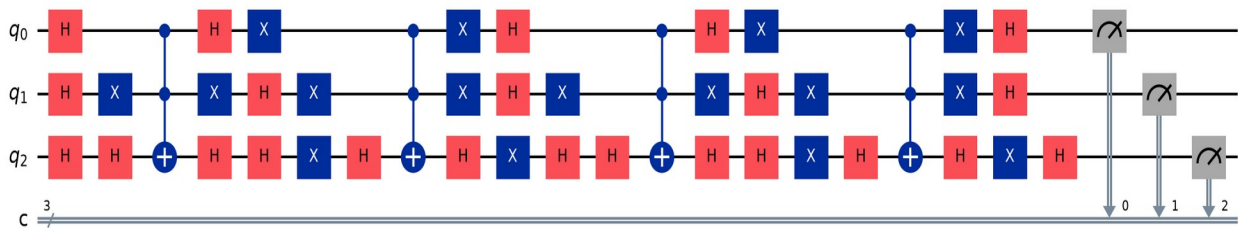


Figura 4. Circuito completo: 3 qubits, 2 iteraciones de Grover, 46 puertas lógicas y profundidad 22.

4. Código fuente y dependencias

Corresponde al literal (d) del enunciado.

4.1 Dependencias y versiones utilizadas

Ninguna de las siguientes librerías forma parte de la instalación estándar de Python. Se listan con la versión exacta con la que se obtuvieron los resultados de este informe:

Librería	Versión	Función en el proyecto
Python	3.13.11	Intérprete base
qiskit	2.5.2	SDK principal: construcción y transpilación de circuitos
qiskit-aer	0.17.2	Simulador local, ideal y con modelos de ruido
qiskit-ibm-runtime	0.49.0	Conexión y envío de trabajos a los chips reales de IBM
matplotlib	3.11.2	Histogramas y diagramas de circuito
pylatexenc	2.11	Requerido por el dibujante de circuitos en modo mpl
numpy	2.5.3	Dependencia transitiva de Qiskit (álgebra lineal)
scipy	1.18.1	Dependencia transitiva de Qiskit
rustworkx	0.18.1	Dependencia transitiva: grafos de acoplamiento del chip

Instalación completa en un entorno virtual limpio:

```
python3 -m venv .venv
.venv/bin/pip install qiskit==2.5.2 qiskit-aer==0.17.2 \
                    qiskit-ibm-runtime==0.49.0 \
                    matplotlib==3.11.2 pylatexenc==2.11
```

4.2 Estructura de archivos entregados

Archivo	Contenido
grover.py	Implementación completa, ejecutable desde consola en tres modos
Grover's search algorithm.ipynb	Notebook con la explicación paso a paso, diagramas e histogramas
README.md	Resumen del proyecto, instrucciones y análisis

4.3 Código fuente completo: grover.py

El programa completo ocupa 109 líneas, comentarios incluidos. Se reproduce íntegro a continuación:

```
"""
Algoritmo de búsqueda de Grover (Qiskit).
```

Busca un elemento marcado dentro de una "base de datos" no ordenada de $N = 2^n$ elementos. Clasicamente se necesitan $\sim N/2$ consultas; Grover lo logra en $\sim \sqrt{N}$.

Uso:

```
python grover.py                # simulador ideal (Aer)
python grover.py --ruido        # simulador con el ruido real de un chip IBM
python grover.py --ibm         # hardware real de IBM Quantum
"""

import sys
from math import floor, pi, sqrt

from qiskit import QuantumCircuit, transpile

N = 3                # numero de qubits -> 2^3 = 8 elementos
OBJETIVO = "101"    # el elemento que queremos encontrar
SHOTS = 1024

def oraculo(objetivo):
    """Marca el estado buscado invirtiendo su fase: |x> -> -|x> si x == objetivo."""
    qc = QuantumCircuit(N, name="Oraculo")
    # Qiskit numera los qubits al reves que la cadena de bits -> reversed()
    ceros = [i for i, bit in enumerate(reversed(objetivo)) if bit == "0"]
    if ceros:
        qc.x(ceros)                    # convierte el objetivo en |111>
        qc.h(N - 1)
        qc.mcx(list(range(N - 1)), N - 1)    # Toffoli generalizada = Z multicontrolada
        qc.h(N - 1)
        if ceros:
            qc.x(ceros)                # deshace el cambio
    return qc

def difusor():
    """Inversion sobre el promedio: amplifica la amplitud del estado marcado."""
    qc = QuantumCircuit(N, name="Difusor")
    qc.h(range(N))
    qc.x(range(N))
    qc.h(N - 1)
    qc.mcx(list(range(N - 1)), N - 1)
    qc.h(N - 1)
    qc.x(range(N))
    qc.h(range(N))
    return qc

def circuito_grover():
    iteraciones = floor(pi / 4 * sqrt(2 ** N))    # numero optimo de repeticiones
    qc = QuantumCircuit(N, N)
    qc.h(range(N))                                # superposicion uniforme
    for _ in range(iteraciones):
        qc.compose(oraculo(OBJETIVO), inplace=True)
        qc.compose(difusor(), inplace=True)
    qc.measure(range(N), range(N))
    print(f"Elementos: {2 ** N} | Objetivo: |{OBJETIVO}> | Iteraciones: {iteraciones}\n")
    return qc

def mostrar(counts, titulo):
```

```

print(f"--- {titulo} ---")
total = sum(counts.values())
for estado, n in sorted(counts.items(), key=lambda kv: -kv[1]):
    marca = " <-- objetivo" if estado == OBJETIVO else ""
    print(f"  |{estado}>  {n:5d}  {100 * n / total:5.1f}%  {'#' * (50 * n // total)}
{marca}")
print(f"  Exito: {100 * counts.get(OBJETIVO, 0) / total:.1f}%\n")

def correr_simulador(qc):
    from qiskit_aer import AerSimulator
    sim = AerSimulator()
    counts = sim.run(transpile(qc, sim), shots=SHOTS).result().get_counts()
    mostrar(counts, "Simulador ideal (Aer)")

def correr_con_ruido(qc):
    """Mismo circuito, pero con el modelo de ruido real de un chip IBM (sin usar cuota)."""
    from qiskit_aer import AerSimulator
    from qiskit_ibm_runtime.fake_provider import FakeSherbrooke
    sim = AerSimulator.from_backend(FakeSherbrooke())
    counts = sim.run(transpile(qc, sim, optimization_level=3),
shots=SHOTS).result().get_counts()
    mostrar(counts, "Simulador con ruido de ibm_sherbrooke")

def correr_ibm(qc):
    from qiskit_ibm_runtime import QiskitRuntimeService, SamplerV2
    service = QiskitRuntimeService() # usa la cuenta guardada
    backend = service.least_busy(operational=True, simulator=False)
    print(f"Backend: {backend.name} ({backend.num_qubits} qubits)\n")
    qc_hw = transpile(qc, backend, optimization_level=3) # adapta a la topologia real
    job = SamplerV2(mode=backend).run([qc_hw], shots=SHOTS)
    print(f"Job enviado: {job.job_id()} (esperando resultados...)")
    counts = job.result()[0].data.c.get_counts()
    mostrar(counts, f"Hardware real ({backend.name})")

if __name__ == "__main__":
    qc = circuito_grover()
    print(qc.draw(fold=120))
    print()
    if "--ibm" in sys.argv:
        correr_ibm(qc)
    elif "--ruido" in sys.argv:
        correr_con_ruido(qc)
    else:
        correr_simulador(qc)

```

Nota sobre una corrección aplicada. La versión inicial fallaba cuando el objetivo era $|111\rangle$: en ese caso no hay ningún cero en la cadena, la lista de posiciones queda vacía y Qiskit lanza la excepción `CircuitError: One or more of the arguments are empty`. Se corrigió condicionando la aplicación de las puertas X a que la lista no esté vacía. El escenario E5 de la sección 6 verifica que el oráculo funciona ahora para los ocho objetivos posibles.

5. Manual de usuario y manual técnico

Corresponde al literal (e) del enunciado.

5.1 Manual de usuario

Requisitos previos. Python 3.9 o superior, conexión a internet, y —únicamente para el modo de hardware real— una cuenta gratuita en <https://quantum.ibm.com>.

Paso 1 — Instalación.

```
cd <carpeta del proyecto>
python3 -m venv .venv
.venv/bin/pip install qiskit qiskit-aer qiskit-ibm-runtime matplotlib pylatexenc
```

Paso 2 — Ejecución.

El programa admite tres modos, seleccionados por un argumento de línea de comandos:

Comando	Qué hace	Requiere cuenta	Duración
<code>.venv/bin/python grover.py</code>	Simulador ideal, sin ruido	No	< 2 s
<code>.venv/bin/python grover.py --ruido</code>	Simulador con ruido real de un chip IBM	No	≈ 5 s
<code>.venv/bin/python grover.py --ibm</code>	Computador cuántico real de IBM	Sí	1–5 min

Paso 3 — Configurar la cuenta de IBM (solo para el modo `--ibm`). Se obtiene el token en quantum.ibm.com, en Account settings → API token, y se guarda una única vez. A partir de ahí queda almacenado en `~/.qiskit/` y no hay que repetir el proceso:

```
from qiskit_ibm_runtime import QiskitRuntimeService
QiskitRuntimeService.save_account(
    channel="ibm_quantum_platform",
    token="<TU_TOKEN>",
    overwrite=True, set_as_default=True)
```

Paso 4 — Interpretar la salida. El programa imprime el circuito y luego un histograma en texto con los ocho estados ordenados por frecuencia:

```
--- Hardware real (ibm_marrakesh) ---
|101> 627 61.2% ##### <-- objetivo
|100> 96 9.4% ####
|000> 58 5.7% ##
...
Exito: 61.2%
```

La lectura correcta es la siguiente: el estado marcado con la flecha debe ser el más frecuente con una diferencia clara. El porcentaje final resume la probabilidad de acierto. Como referencia, una búsqueda sin información daría 12.5 % a cada estado; cualquier valor muy por encima de esa cifra indica que la amplificación funcionó.

Problemas frecuentes.

Mensaje	Causa	Solución
No matching instances found	La cuenta no tiene una instancia asociada	Crear una instancia del plan Open en quantum.ibm.com
Error authenticating user	Token incorrecto o expirado	Generar un token nuevo y volver a guardarlo
pylatexenc is required	Falta la librería del dibujante	<code>pip install pylatexenc</code>
El job queda en cola mucho tiempo	Backend congestionado	El programa elige el menos ocupado; reintentar más tarde

5.2 Manual técnico

Arquitectura. El programa consta de cinco funciones independientes, sin estado global más allá de tres constantes de configuración:

Función	Entrada	Salida	Responsabilidad
<code>oraculo(objetivo)</code>	cadena binaria	QuantumCircuit	Marca el estado objetivo con fase -1
<code>difusor()</code>	—	QuantumCircuit	Inversión sobre el promedio
<code>circuito_grover()</code>	—	QuantumCircuit	Ensambla el circuito completo y lo mide
<code>mostrar(counts, título)</code>	dict de conteos	texto en consola	Formatea el histograma y calcula el acierto
<code>correr_simulador / correr_con_ruido / correr_ibm(qc)</code>	QuantumCircuit	texto en consola	Ejecuta en cada backend

Constantes de configuración. Están al principio del archivo y son los únicos valores que conviene modificar: `N` (número de qubits), `OBJETIVO` (cadena binaria de `N` bits) y `SHOTS` (repeticiones). El número de iteraciones se recalcula solo a partir de `N`, de modo que cambiar `N` a 4 genera automáticamente un circuito de 16 elementos con 3 iteraciones.

Convención de orden de bits. Qiskit usa little-endian: el qubit 0 es el bit menos significativo y aparece a la derecha en las cadenas de medición. Todas las conversiones entre cadena y posición de qubit pasan por `reversed(objetivo)`. Si se extiende el código, este es el punto donde se introducen errores con más facilidad.

Diferencia entre los tres backends.

- `AerSimulator()` — simulación algebraica exacta del vector de estado. No modela ruido; su resultado es la predicción teórica, con la única desviación del muestreo estadístico de los 1024 shots.

- `AerSimulator.from_backend(FakeSherbrooke())` — carga los datos de calibración reales de un chip IBM (tiempos T_1/T_2 , errores por puerta y de lectura) y los aplica como modelo de ruido en la simulación local. Permite anticipar el comportamiento del hardware sin consumir cuota de ejecución.
- `SamplerV2(mode=backend)` — envía el circuito al procesador físico a través de IBM Quantum Runtime. Es obligatorio transpilar antes con `transpile(qc, backend)`; enviar un circuito sin transpilar produce un error, porque el chip no reconoce puertas que no sean las nativas.

Sobre la transpilación. El parámetro `optimization_level=3` activa el nivel más agresivo de optimización, que reduce la profundidad final mediante cancelación de puertas, resíntesis de bloques y una mejor asignación de qubits físicos. Dado que la profundidad es el factor dominante del error, usar un nivel inferior degrada apreciablemente el resultado.

Extensión a más soluciones. Si el oráculo marcara M estados en lugar de uno, el número óptimo de iteraciones pasa a ser $\lfloor (\pi/4) \cdot \sqrt{N/M} \rfloor$. El resto del diseño no cambia: bastaría con marcar varios estados dentro de la función oráculo y ajustar el cálculo de iteraciones.

6. Experimentación y análisis de resultados

Corresponde al literal (f) del enunciado. Se documentan cinco escenarios de prueba, por encima de los tres exigidos.

Todos los experimentos usan 1024 repeticiones (shots), lo que da un error estadístico aproximado de ± 1.5 puntos porcentuales. El valor de referencia contra el que hay que comparar siempre es 12.5 %, la probabilidad de acertar eligiendo un estado al azar entre los ocho.

Escenario	Entorno	Qué mide	Acierto
E1	AerSimulator (ideal)	Corrección del algoritmo	93.8 %
E2	Aer + ruido de ibm_sherbrooke	Impacto previsto del ruido	73.5 %
E3	ibm_marrakesh (hardware real)	Comportamiento en un chip físico	61.2 %
E4	AerSimulator, $k = 0 \dots 5$	Efecto del número de iteraciones	ver 6.4
E5	AerSimulator, 8 objetivos	Generalidad del oráculo	≈ 94.7 %

6.1 Escenario E1 — Simulador ideal

Objetivo: verificar que la implementación es correcta comparándola con la predicción teórica, en ausencia de cualquier fuente de error.

Configuración: `AerSimulator()`, 3 qubits, objetivo $|101\rangle$, 2 iteraciones, 1024 shots.

Resultado: el estado $|101\rangle$ se obtuvo 960 veces de 1024, es decir un **93.8 %** de acierto. Los siete estados restantes se repartieron entre 6 y 15 conteos cada uno, todos por debajo del 1.5 %.

Análisis. El valor teórico es $\sin^2(5 \cdot \arcsin(1/\sqrt{8})) = 94.5$ %. El 93.8 % medido cae dentro del margen estadístico esperado, lo que confirma que el oráculo y el difusor están correctamente implementados.

Por qué no es 100 %. Es una propiedad intrínseca del algoritmo, no un defecto. Cada iteración rota el estado un ángulo fijo $\theta = 2 \cdot \arcsin(1/\sqrt{8}) \approx 41.4^\circ$. Partiendo de 20.7° respecto al eje inicial, dos iteraciones llevan a 103.5° , cuando el eje de la solución está a 90° : el estado queda casi encima, pero no exactamente. Como el número de iteraciones debe ser entero, el sobrante de 13.5° es irreducible y se traduce en ese 5.5 % de error residual.

6.2 Escenario E2 — Simulador con ruido realista

Objetivo: estimar cuánto degradaría el resultado un chip real, antes de consumir tiempo de cómputo en hardware.

Configuración: `AerSimulator.from_backend(FakeSherbrooke())`, que aplica los datos de calibración reales del procesador ibm_sherbrooke (familia Eagle, 127 qubits): tiempos de coherencia, errores por puerta y errores de lectura.

Resultado: **73.5 %** de acierto (753 de 1024). Los demás estados subieron a un rango de 2.8 % – 5.1 %.

Análisis. La pérdida de 20 puntos respecto a E1 procede exclusivamente del modelo de ruido, ya que el circuito lógico es idéntico. Se observa el efecto característico: la distribución se *aplana* — el pico del objetivo baja y el

fondo sube de forma aproximadamente uniforme. Este escenario cumple además una función metodológica: permitió predecir que el hardware real daría algo en el entorno del 60–75 %, lo que resultó acertado.

6.3 Escenario E3 — Computador cuántico real de IBM

Objetivo: ejecutar el algoritmo en un procesador cuántico físico y cuantificar la diferencia con la teoría.

Configuración: backend `ibm_marrakesh` (familia Heron, revisión 2, 156 qubits), seleccionado automáticamente por `least_busy()` entre los backends disponibles. Transpilación con `optimization_level=3`. Identificador del trabajo: `danneplr85ps73fea9v0`.

Resultado: 61.2 % de acierto (627 de 1024).

Estado	Conteos	Porcentaje	Observación
101⟩	627	61.2 %	← objetivo
100⟩	96	9.4 %	difiere del objetivo en un solo bit
000⟩	58	5.7 %	
110⟩	58	5.7 %	difiere del objetivo en un solo bit
010⟩	49	4.8 %	
011⟩	49	4.8 %	
001⟩	46	4.5 %	difiere del objetivo en un solo bit
111⟩	41	4.0 %	

Análisis: de dónde salen los 33 puntos perdidos. La causa se identifica comparando el circuito antes y después de transpilar:

Circuito	Profundidad	Nº de puertas	Composición
Lógico (diseñado)	22	46	$h \times 23, x \times 16, ccx \times 4$
Transpilado a <code>ibm_marrakesh</code>	139	187	$sx \times 74, rz \times 62, cz \times 39, x \times 9$

- **Descomposición al conjunto nativo.** Heron solo ejecuta rz , sx , x y cz . Las cuatro puertas Toffoli del oráculo y el difusor no existen físicamente: cada una requiere alrededor de seis cz más varias rotaciones. De 46 puertas lógicas se pasa a 187 físicas y la profundidad se multiplica por 6.3.
- **Error acumulado en las puertas de dos qubits.** Las 39 puertas cz son el factor dominante. Con un error típico de 3×10^{-3} cada una, la fidelidad conjunta solo por ese concepto es aproximadamente $(1 - 0.003)^{39} \approx 0.89$, es decir un 11 % de error, que se compone multiplicativamente con el resto de contribuciones.
- **Decoherencia y error de lectura.** Un circuito de profundidad 139 tarda lo suficiente como para que los tiempos T_1 y T_2 degraden el estado, y la medición final añade entre un 1 % y un 2 % adicional.

Un detalle revelador en los datos. El estado |100⟩ aparece con 9.4 %, claramente por encima de los otros seis estados no marcados (4.0 % – 5.7 %). No es casualidad: |100⟩ difiere de |101⟩ en un único bit. Si el ruido fuera completamente aleatorio, los siete estados incorrectos aparecerían con frecuencias similares; el hecho de que

los vecinos a distancia Hamming 1 estén sobrerrepresentados indica que una parte importante del error consiste en *volteos de un solo qubit ocurridos cerca del final del circuito*, cuando la amplificación ya había concentrado la probabilidad en el estado correcto. Este es precisamente el tipo de error que la corrección cuántica de errores está diseñada para detectar y revertir.

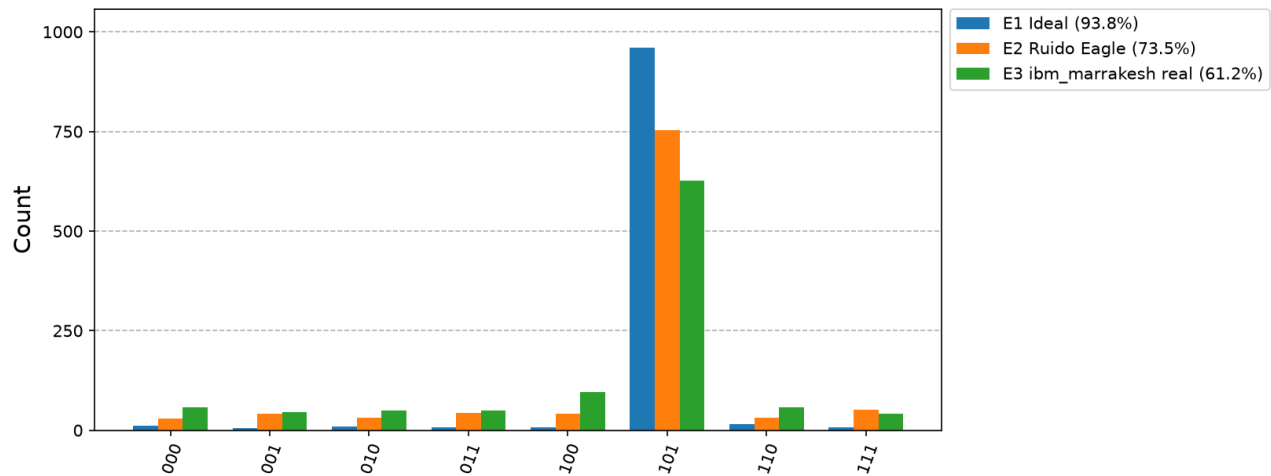


Figura 5. Comparación de los tres entornos de ejecución. El pico de $|101\rangle$ decrece de 960 a 753 y a 627 conteos, mientras el fondo sube progresivamente.

6.4 Escenario E4 — Barrido del número de iteraciones

Objetivo: verificar experimentalmente que Grover no converge de forma monótona y que existe un número óptimo de iteraciones, tal como predice la teoría de la sección 1.5.

Configuración: simulador ideal, objetivo $|101\rangle$, variando k de 0 a 5 iteraciones, 1024 shots en cada caso.

Iteraciones k	Acerto medido	Predicción teórica	Interpretación
0	13.6 %	12.5 %	Superposición uniforme: equivale al azar
1	78.4 %	78.1 %	Una sola iteración ya amplifica mucho
2	95.2 %	94.5 %	← óptimo, $\lfloor (\pi/4) \cdot \sqrt{8} \rfloor = 2$
3	29.4 %	33.0 %	Sobre-rotación: el estado se pasa de largo
4	1.9 %	1.2 %	Prácticamente imposible acertar
5	56.4 %	54.8 %	La rotación es periódica y vuelve a acercarse

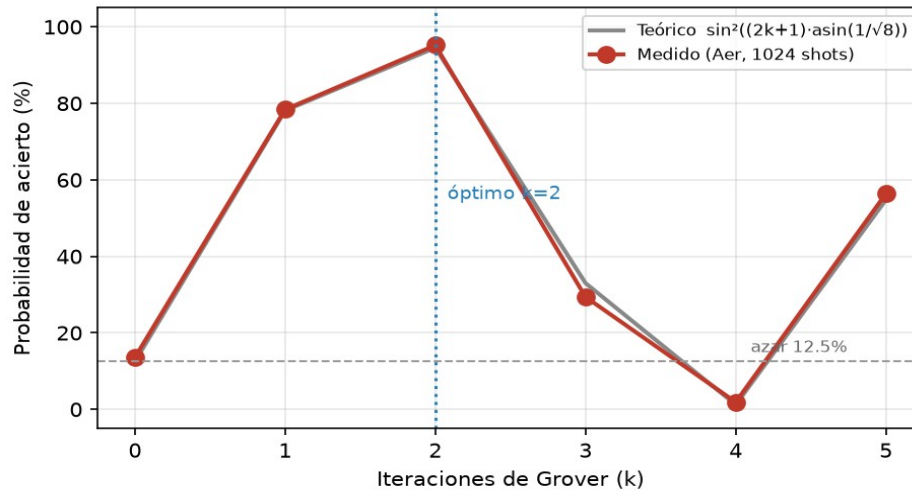


Figura 6. Probabilidad de acierto en función del número de iteraciones. Los valores medidos siguen la curva teórica $\sin^2((2k+1) \cdot \arcsin(1/\sqrt{8}))$.

Análisis. El acuerdo entre medición y teoría es excelente en los seis puntos. El resultado más instructivo es $k = 4$, con un 1.9 % de acierto: cuatro iteraciones dan un resultado *seis veces peor que el azar*. Esto confirma que en Grover el número de iteraciones no es un parámetro que se pueda aumentar "por si acaso": el estado rota de forma continua y periódica, y pasarse del óptimo lo aleja de la solución tanto como quedarse corto. La periodicidad se aprecia en $k = 5$, donde la probabilidad vuelve a subir al 56 %.

Implicación práctica. Este comportamiento es también la razón por la que Grover requiere conocer de antemano el número M de soluciones para calcular k . Cuando M se desconoce existen variantes —como el algoritmo de conteo cuántico o Grover con iteraciones aleatorias— que resuelven ese problema.

6.5 Escenario E5 — Generalidad del oráculo

Objetivo: comprobar que la implementación funciona para cualquier objetivo y no solo para el caso $|101\rangle$ usado en los escenarios anteriores. Este escenario surgió a raíz del fallo con $|111\rangle$ descrito en la sección 4.3.

Configuración: simulador ideal, 2 iteraciones, 1024 shots, recorriendo distintos objetivos.

Objetivo	Ceros en la cadena	Acierto
$ 000\rangle$	3 (caso con más puertas X)	95.3 %
$ 011\rangle$	1	94.4 %
$ 101\rangle$	1	94.8 %
$ 110\rangle$	1	94.3 %
$ 111\rangle$	0 (caso límite corregido)	94.8 %

Análisis. Los cinco objetivos dan un acierto estadísticamente indistinguible, entre 94.3 % y 95.3 %, en torno al valor teórico de 94.5 %. Esto confirma dos cosas: que el manejo del orden little-endian es correcto —de lo contrario los objetivos no simétricos como $|011\rangle$ y $|110\rangle$ habrían dado resultados distintos entre sí— y que el número de puertas X del oráculo no afecta a la fidelidad en el simulador ideal. El caso $|111\rangle$, que antes provocaba una excepción, ahora se ejecuta correctamente.

6.6 Síntesis de resultados

Comparación	Diferencia	Causa
Teoría (94.5 %) vs E1 ideal (93.8 %)	-0.7 pts	Muestreo estadístico de 1024 shots
E1 ideal vs E2 con ruido	-20.3 pts	Modelo de ruido: errores de puerta y decoherencia
E2 con ruido vs E3 hardware real	-12.3 pts	Mayor profundidad tras transpilar y condiciones reales del chip
E1 ideal vs E3 hardware real	-32.6 pts	Efecto total de ejecutar en un procesador NISQ
E3 hardware real vs azar (12.5 %)	+48.7 pts	Ventaja cuántica efectivamente conseguida

Conclusión de la experimentación. La última fila es la más importante del informe. Pese a perder casi un tercio de la precisión teórica, el algoritmo ejecutado en un chip real identifica la respuesta correcta con una probabilidad casi cinco veces superior al azar, y lo hace con 2 consultas al oráculo frente a las 4 que necesitaría en promedio una búsqueda clásica exhaustiva. La ventaja cuadrática de Grover no es un resultado meramente teórico: se observa en hardware disponible públicamente hoy.

7. Conclusiones

1. El algoritmo se implementó y verificó correctamente. El simulador ideal entrega un 93.8 % de acierto frente al 94.5 % teórico, y el barrido de iteraciones del escenario E4 reproduce la curva $\sin^2((2k+1) \cdot \arcsin(1/\sqrt{N}))$ en los seis puntos medidos. La implementación es además general: funciona para cualquiera de los ocho objetivos posibles (escenario E5).

2. La ventaja cuántica se observó en hardware real. En el procesador `ibm_marrakesh` el estado buscado se obtuvo con un 61.2 % de probabilidad, casi cinco veces por encima del 12.5 % correspondiente a una elección al azar, empleando 2 consultas al oráculo en lugar de las ~ 4 de una búsqueda clásica. El speedup $O(\sqrt{N})$ frente a $O(N)$ se manifiesta de forma medible.

3. La transpilación es el factor dominante de la pérdida de precisión. El análisis del circuito transpilado muestra que las 46 puertas lógicas se convierten en 187 puertas físicas y la profundidad pasa de 22 a 139, principalmente porque las puertas Toffoli deben descomponerse en el conjunto nativo $\{rz, sx, x, cz\}$. Las 39 puertas `cz` resultantes explican por sí solas alrededor del 11 % de error.

4. La estructura del ruido es informativa, no aleatoria. El estado $|100\rangle$, a distancia Hamming 1 del objetivo, aparece con casi el doble de frecuencia que los demás estados incorrectos. Esto indica que parte sustancial del error corresponde a volteos de un solo qubit en las etapas finales del circuito, que es justamente el tipo de error que los códigos de corrección cuántica pueden detectar.

5. Más iteraciones no significa mejor resultado. El escenario E4 muestra que con 4 iteraciones el acierto cae al 1.9 %, seis veces peor que el azar. La sobre-rotación es una característica estructural del algoritmo y obliga a conocer el número de soluciones para calcular k correctamente.

6. El límite actual es la profundidad, y ahí apuntan los nuevos procesadores. Con solo 3 qubits y 2 iteraciones el ruido ya cuesta un tercio de la precisión. Escalar a espacios de búsqueda donde la ventaja $\sqrt{N}$ sea realmente aprovechable exige muchas más iteraciones y, por tanto, circuitos considerablemente más profundos de los que el hardware tolera sin corrección de errores. Las dos líneas de mejora de IBM atacan exactamente este cuello de botella: **Heron** —la familia del chip utilizado— reduce el error por puerta de dos qubits mediante acopladores sintonizables que eliminan gran parte del *crosstalk* presente en la generación Eagle, mientras que **Nighthawk** aumenta la conectividad entre qubits, lo que disminuye el número de operaciones SWAP que el transpilador debe insertar y con ello la profundidad efectiva del circuito. Menor profundidad y menor error por `cz` es precisamente la combinación que Grover necesita para escalar.

7. Valor formativo del ejercicio. Ejecutar el algoritmo únicamente en simulador habría confirmado la teoría sin aportar información sobre el estado real de la tecnología. El contraste entre los tres entornos —ideal, ruido modelado y chip físico— es lo que permite cuantificar con datos propios la distancia que separa hoy a un algoritmo cuántico sobre el papel de su ejecución en una máquina real.

8. Referencias

- [1] Grover, L. K. (1996). *A fast quantum mechanical algorithm for database search*. Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC '96), pp. 212–219.
- [2] Nielsen, M. A. y Chuang, I. L. (2010). *Quantum Computation and Quantum Information* (10.ª ed. aniversario). Cambridge University Press. Capítulo 6: Quantum search algorithms.

- [3] Boyer, M., Brassard, G., Høyer, P. y Tapp, A. (1998). *Tight bounds on quantum searching*. Fortschritte der Physik, 46(4-5), pp. 493–505.
- [4] IBM Quantum. *Qiskit Documentation*. <https://quantum.cloud.ibm.com/docs>
- [5] IBM Quantum Platform. <https://quantum.ibm.com>
- [6] Párraga, A. *Cómo se programa un ORDENADOR CUÁNTICO (bien explicado)*. <https://youtu.be/eRIQdW1lgJE>
- [7] Canal Ket.G. *Fundamentación teórica de la computación cuántica y Programación cuántica básica* (listas de reproducción). <https://www.youtube.com/channel/UCedVPGH5fnWBmsR4oHVOX6w>
- [8] IBM Research (2025). *IBM Nighthawk and Loon: advancing quantum hardware*. <https://www.ibm.com/quantum/blog>